Advanced RAG Architectures

A teaching companion — Week 7, Session 1

THE FRAMING

Why this session opens the week

In Week 3, students built naive RAG (chunk → embed → retrieve → stuff into prompt) and were happy when it returned *something*. Session 1 of Week 7 reframes the question. The line from the deck is:

> *"We stop asking does it return a chunk? and start asking does it return the right chunk, can we prove it, and does it survive when the question is hard?"*

That reframe sets up the whole week:

- **S1** attacks the *right chunk* half.
- **S2** (Re-Ranking & GraphRAG) attacks the *prove it* half through ranking quality.
- **S3** (Enterprise Connectors & Ingestion) is the unglamorous plumbing underneath all of it.
- **S4** (RAG Evaluation) attacks the proof itself.

THE BASELINE

Why naive RAG breaks

Before introducing solutions, the session names four failure modes students will recognize from their own Week 3 projects. Each technique introduced later maps back to one of these. **That is the pedagogical anchor** — students should be able to point at a technique and name the failure it fixes.

1. **Chunk size tyranny.** Small chunks lose context. Big chunks dilute relevance. One size fits no question.
2. **Vocabulary mismatch.** Users ask "how do I cancel?" but the doc says "termination procedure." Cosine similarity shrugs.
3. **Top-k is a guess.** k=5 returns 5 chunks by similarity — not necessarily the 5 you need.
4. **Multi-hop questions die.** "Compare the refund policy with the cancellation policy" — one vector search will not retrieve both.

PART 01

Multi-vector & parent-document retrieval

The first lever: stop indexing only the chunk you want to return.

Multi-vector retrieval

Index *multiple representations* of the same document (small sub-chunks, summaries, hypothetical questions), but *return the original*. The query searches against whichever representation matches it best — short queries find sub-chunks, abstract queries find summaries, FAQ-style queries find hypothetical questions.

LangChain's `MultiVectorRetriever` uses two storage layers: a `vectorstore` for the searchable representations, a `docstore` for the parents, linked by a shared `id_key`.

> **THE MENTAL MODEL TO DRIVE HOME**
>
> You **search** against the version that matches the query best. You **return** the version that gives the LLM the most useful context. These do not have to be the same thing — that is the whole point.

Parent-document retrieval

A specialization of the above. Use a small `child_splitter` (~400 chars) for what gets embedded and searched. Use a big `parent_splitter` (~2000 chars) for what gets returned. **Match small, return big.** This is the default starting point for production — handles the precision/context tradeoff with one config change.

When to reach for which

- Default: **parent-document retrieval**. Cheap to set up, handles most cases.
- Move to **multi-vector with summaries** only when users ask high-level questions of detailed documents.
- Move to **multi-vector with hypothetical questions** only when the corpus and the questions speak different languages (FAQs, jargon-heavy domains).

> **THE RULE OF THUMB**
>
> Don't reach for the more complex tool until you can name the failure pattern it solves. "Multi-vector sounds cool" isn't a reason to ship it.

PART 02

HyDE & query transformations

The second lever: reshape the *query* rather than the index.

HyDE — Hypothetical Document Embeddings

This is the one students remember. The trick, in three steps:

1. Ask the LLM to write a hypothetical answer to the question — let it hallucinate freely.
2. Embed the hallucinated answer (**not** the question) and search the vector store with it.
3. Return the real documents. Throw the hallucination away — you only used its shape.

Why it works

The asymmetry problem: a short question and a long answer don't share much vocabulary, so cosine similarity underweights the right doc even when it's sitting there. A hallucinated answer is *shaped like* the real answer, so it lands closer in embedding space.

When to reach for it

- Short, abstract queries.
- Jargon mismatch between users and corpus.
- Single-doc factual retrieval where the answer is short.

What to watch for

- Adds an LLM call per query — latency and cost.
- Drifts on niche domains the LLM doesn't know well.
- Can amplify bias if the hallucination is wrong in interesting ways.

Other query transformations

Introduced in the deck as a card grid — students should know what each is for:

- **Multi-query.** LLM rewrites the question 3–5 ways, union of all retrieved chunks. Helps with phrasing variance.
- **Step-back prompting.** LLM asks a more general question first, retrieves broad context, then narrows.
- **Query decomposition.** Break multi-hop questions into atomic sub-questions, retrieve for each, combine.
- **Routing.** Classify the query and send it to the right vector store / tool / index. Foundation for agentic RAG.

> **ONE TEACHING NOTE**
>
> The multi-query code slide is the worked example. The explicit instruction: **log the generated queries.** Without that, debugging multi-query retrieval is guesswork.

PART 03

Agentic RAG patterns

The third lever: stop treating retrieval as a fixed step.

From pipeline to agent

The conceptual hinge of Part 03. **Naive RAG is linear:** Query → Retrieve → Stuff → Generate. Always retrieves, always stuffs, always answers — even when it shouldn't.

Agentic RAG turns the diagram into a loop: Query → Plan/Route → Retrieve (as a tool call) → Reflect/Critique → Answer or retry. The system decides *whether* to retrieve, *what* to retrieve, and whether the answer is good enough.

Three patterns to know

1. Routing RAG

Classify the query, send it to the right index or tool. Cheapest agentic upgrade — one extra LLM call. **Example:** sales docs vs engineering docs vs HR docs, with different vector stores and prompts per route.

2. Self-correcting RAG

Retrieve, generate, grade. If retrieval was bad → re-query. If the answer is unsupported → retry with feedback. **Examples:** Corrective RAG (CRAG), Self-RAG. **Tradeoff:** 2–3× latency.

3. Tool-using RAG agent

Retrieval is one tool among many. The LLM picks: vector search, SQL, web search, calculator. Built on LangGraph.

> **THE WEEK 4 CALLBACK — SAY THIS OUT LOUD**
> "You already built the agent loop. Agentic RAG is just that loop with retrieval as a tool. No new framework — same LangGraph muscles." Students who survived Week 4 already have the mental model. What looks like a new topic is the agent pattern applied to a familiar problem.

The code slide

A minimal LangGraph sketch:

- A `RAGState` TypedDict.
- A `retrieve` node.
- A `grade_docs` node that returns `{"route": "answer" | "retry"}`.
- A `rewrite_query` node.
- `add_conditional_edges` on the grade node's route.

Enough to understand the shape without a full implementation in the deck.

THE TAKEAWAY

The decision framework

The penultimate slide is the practical takeaway — a ladder of failure modes to fixes. **Order matters:** adding HyDE on top of bad retrieval just hides the real problem. Fix the floor first.

If you see this	Try this
Retrieval misses the right chunk most of the time	**Parent-document or multi-vector first**
Right chunks come back, but ranked too low	**Re-ranking — that's S2**
Queries don't share vocabulary with the corpus	**HyDE or multi-query rewriting**
Questions span multiple topics or hops	**Query decomposition or agentic routing**
System should sometimes skip retrieval entirely	**Full agentic RAG with retrieval as a tool**

THE WARNING TO REPEAT

Order matters. Adding HyDE on top of bad retrieval just hides the real problem. Diagnose the failure mode first, then pick the lever. Stacking techniques without diagnosis is how systems get slower and worse at the same time.

THE HANDS-ON

What students do after the session

The session closes with a five-step challenge that locks the learning into student work:

1. Pick 20 hard questions your current Week 3 RAG fails on. Save them.
2. Re-build with parent-document retrieval. Compare top-k on the same 20.
3. Add HyDE in front of it. Compare again.
4. Pick **one** failure mode you still see. Name it.
5. Bring that named failure to Session 2 — re-ranking is where it goes.

WHY STEP 5 MATTERS

That last step is what threads S1 into S2 — each session ends with the work that becomes the input to the next. Students don't arrive at S2 cold; they arrive carrying a specific, named failure that S2 was built to address.